Layer 4 Attacks and Defenses

IE3122: Network Security | Week 6

TCP, UDP, and what happens when the transport layer is abused

Today's Agenda

- Layer 4 fundamentals: mandatory tasks, and what TCP adds beyond them
- TCP versus UDP, compared function by function
- The concept of a socket
- TCP connection establishment and teardown, and what resources get reserved at each stage
- Six TCP-based attacks: port scans, host sweeps, OS fingerprinting, hijacking, RST injection, SYN flood
- How lightweight UDP is, and four UDP-based attacks
- Bringing it together: recurring vulnerability themes and layering mitigations affordably

Learning Outcomes for Today

LO1

- Describe common network vulnerabilities and attacks, today specifically at the transport layer.

LO2

- Evaluate different mitigation techniques for vulnerabilities and make recommendations.

- *By the end of this session you should be able to explain what TCP and UDP each actually guarantee, how an attacker abuses the gap between the two, and how to layer defenses that are proportionate to cost.*

Layer 4 Fundamentals

What the transport layer is actually responsible for

Mandatory Tasks of Layer 4

- Segmentation: breaking application data into transportable units, small enough to hand down to the network layer.
- Multiplexing and demultiplexing: port numbers let many applications on the same host share one IP address, and let the receiving host route incoming data to the correct application.
- End-to-end delivery between processes: recall that IP only gets a packet from host to host. Layer 4 is what actually delivers it to the right application on that host.
- Basic error detection: both TCP and UDP carry a checksum field, giving a simple way to detect corruption in transit.
- Every protocol at this layer does these five things. What separates TCP from UDP is everything built on top of them.

What TCP Adds Beyond the Mandatory Tasks

- Connection establishment and termination: TCP is stateful. Two ends explicitly agree to start communicating, and explicitly agree to stop.
- Reliability: every segment is acknowledged, and anything lost gets retransmitted automatically.
- Ordered delivery: sequence numbers let the receiver reassemble segments in the correct order, even if they arrive out of order.
- Flow control: a sliding window keeps a fast sender from overwhelming a slow receiver.
- Congestion control: TCP actively adjusts its sending rate in response to network conditions, protecting the network itself, not just the two endpoints.
- UDP does none of these. That's not a flaw, it's a deliberate design choice, and it's exactly what today's second half is about.

TCP vs UDP: Function by Function

Function	TCP	UDP
Connection setup	Required (three-way handshake)	None, connectionless
Reliability	Guaranteed via acknowledgment and retransmission	None, best-effort only
Ordering	Guaranteed via sequence numbers	None, packets may arrive in any order
Flow control	Yes, sliding window	None
Congestion control	Yes, actively adjusts sending rate	None
Header size	20 bytes minimum	8 bytes
Typical use	Web, email, file transfer, anything needing correctness	DNS, VoIP, video, gaming, anything needing speed

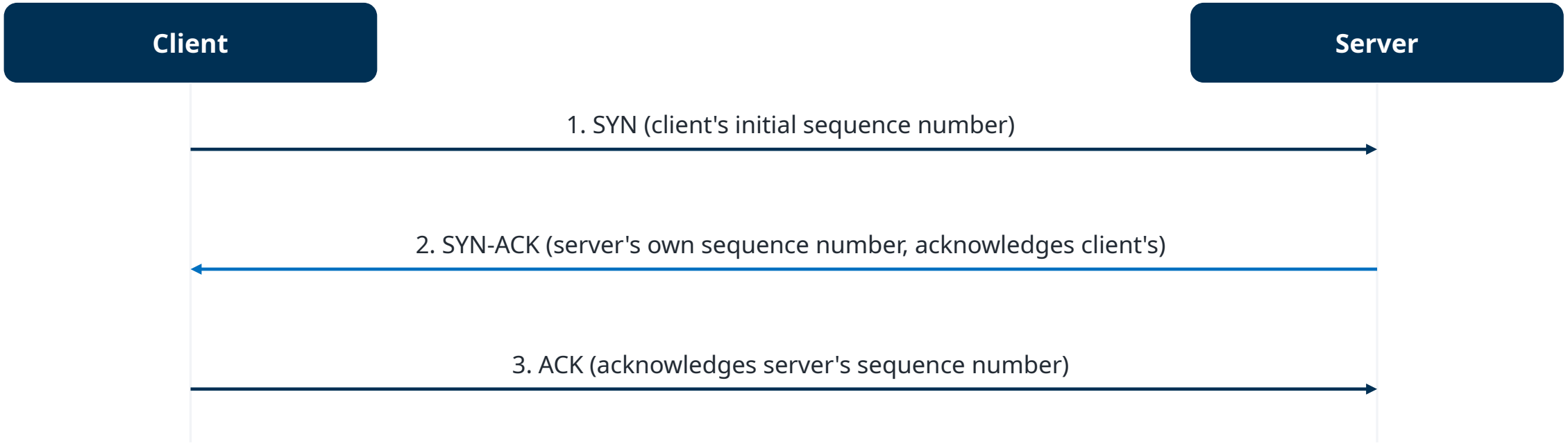
The Concept of a Socket

- A socket is the endpoint of a communication link, identified by an IP address paired with a port number.
- A single connection is uniquely identified by four values together: source IP, source port, destination IP, destination port. This is why your browser can have a dozen tabs all talking to the same web server at once without their traffic getting mixed up. Each tab uses a different source port.
- Operating systems expose sockets through an API (create the socket, bind it to an address, then either listen for incoming connections or connect out to one). You don't need to write this code to understand the module, but recognising the pattern helps when reading how services are configured.
- Every port scan, every session hijack, every SYN flood in today's lecture is ultimately an attack on how a socket is created, tracked, or trusted.

TCP Connection Establishment and Teardown

What gets reserved, and when

The Three-Way Handshake



Both sides pick their own initial sequence number, and the handshake's real job is getting both ends to agree on both numbers before any application data flows.

What Gets Reserved at Each Stage

- When the server receives a SYN, before it has any confirmation the client is real or reachable, it already commits real memory: an entry in its half-open connection backlog, holding the connection's state (a Transmission Control Block).
- The server sends SYN-ACK and waits. The connection sits in this half-open state, consuming a backlog slot, until the final ACK arrives or a timeout expires.
- Only when the final ACK arrives does the server move the connection out of the backlog into the fully established connection table, allocating the real send/receive buffers a working connection needs.
- This is the single most important detail in today's lecture: the server spends real, finite resources on a connection attempt before it has any proof the other side is genuine. Everything about SYN flood, later in this section, follows directly from that asymmetry.

Tearing a Connection Down

- Closing is typically a four-step exchange: the side finishing first sends FIN, the other side acknowledges it, that side then sends its own FIN when it's ready, and the first side sends a final ACK.
- Sometimes the middle two steps combine into a single FIN-ACK if the second side has nothing more to send at the same moment, making it look like a three-way close instead of four.
- After sending that final ACK, the initiating side enters a TIME_WAIT state, holding onto the connection's resources for a defined period (commonly a couple of minutes) before fully releasing them, in case a delayed duplicate segment still shows up.
- This matters operationally, not just academically: a server or client that opens and closes connections very rapidly can accumulate a large number of sockets stuck in TIME_WAIT, which is itself a resource-consumption pattern worth being aware of.

TCP-Based Attacks

Six attacks, each abusing a different part of what you just learned

Six Attacks at a Glance

Attack	Category	What it targets
TCP Port Scans	Information Gathering	Which ports on a host are open, closed, or filtered
TCP Host Sweeps	Information Gathering	Which hosts on a network are alive and reachable
OS Fingerprinting	Information Gathering	Which operating system and version a host is running
TCP Hijacking	Header Abuse	An already-established session between two legitimate parties
TCP RST Injection	Header Abuse	Prematurely terminating a specific existing connection
TCP SYN Flood	Connection Resource Exhaustion	The server's half-open connection backlog

TCP Port Scans: Overview, Vulnerability & Technique

- Overview: the attacker sends TCP packets to a range of ports on a target to determine which have listening services.
- Vulnerability exploited: TCP's handshake behaviour is predictable and reveals port state by design. An open port answers SYN with SYN-ACK; a closed port answers with RST.
- Technique: a SYN scan sends a SYN and reads the reply without ever completing the handshake (quieter, since no full connection is logged). A connect scan completes the full handshake (more detectable, but needs no special privileges). FIN, Xmas, and NULL scans send unusual flag combinations that a closed port should answer with RST, but which many operating systems simply drop on an open port with no reply at all, an ambiguity that can slip past a firewall only watching for SYN packets.
- Impact: reveals exactly which services are worth attacking next. Reconnaissance, not compromise by itself, but almost always the first step toward it.

TCP Port Scans: Mitigation

- Minimise the attack surface first: disable or firewall off any service that doesn't need to be reachable at all. A port that isn't listening can't be scanned into revealing anything.
- Firewall and IDS/IPS detection of scan patterns: a burst of connection attempts across many ports from one source in a short window is a strong, well-known signal.
- Ensure filtered ports behave consistently (a uniform drop, not a mix of drop and reject) so scan results don't leak information about firewall rules themselves.
- Rate-limiting connection attempts per source slows a scan down without necessarily blocking legitimate, occasional traffic.

TCP Host Sweeps: Overview, Vulnerability & Technique

- Overview: rather than scanning every port on one host, the attacker sends a single probe (commonly to a widely-used port like 80) across an entire address range, to find out which hosts are alive at all.
- Vulnerability exploited: hosts respond to TCP probes on an open or closed port even when ICMP itself is blocked, so this is often used specifically to work around a network that has disabled ICMP echo to defeat the simpler sweep from Week 4.
- Technique: send a SYN to the same port across every address in a range; any SYN-ACK or RST confirms a live host, regardless of whether that specific port is actually the attacker's real target.
- Impact: a map of which addresses are worth further investigation, the same reconnaissance value as an ICMP sweep, just via a channel that ICMP filtering alone doesn't stop.

TCP Host Sweeps: Mitigation

- The same core controls as port scanning: minimise exposed services, detect horizontal scan patterns (one port, many destination addresses) at the firewall or IDS, and rate-limit.
- Don't treat ICMP filtering as sufficient reconnaissance defence on its own. This attack exists specifically because ICMP filtering alone isn't enough.
- Network segmentation limits how much of the address space a single sweep from one compromised point can actually reach.

OS Fingerprinting: Overview, Vulnerability & Technique

- Overview: the attacker analyses subtle, characteristic differences in how a target's TCP/IP stack behaves to identify its operating system and version.
- Vulnerability exploited: every OS implements the TCP/IP specification with small, consistent quirks (default TTL, window size, options ordering, exactly how it responds to malformed or unusual packets) that differ enough between operating systems to be identifiable.
- Technique: tools send a series of specially crafted probes and compare the pattern of responses against a database of known OS fingerprints, similar in spirit to how a real fingerprint is matched against a database.
- Impact: knowing the exact OS and version lets an attacker go straight to vulnerabilities known to affect that specific target, a significant force-multiplier for whatever comes next.

OS Fingerprinting: Mitigation

- Keep systems patched. If a specific OS and version is identified but has no outstanding known vulnerability, the fingerprint alone doesn't get the attacker anywhere.
- Some firewalls can normalise or scrub outbound packet characteristics specifically to make fingerprinting less reliable, though this is a minor obstacle rather than a real barrier.
- Treat this as a reason for defence-in-depth generally, not something to solve on its own. The goal is making sure identification doesn't automatically lead to compromise.

TCP Hijacking: Overview, Vulnerability & Technique

- Overview: the attacker takes over an already-established TCP session between two legitimate parties, injecting their own traffic as if they were one of the endpoints.
- Vulnerability exploited: TCP authenticates nothing after the handshake. It only tracks state via sequence numbers, and never cryptographically verifies that later packets actually came from the party who opened the connection.
- Technique: the attacker obtains the current sequence number, either by sniffing traffic on a shared or compromised network segment, or historically by predicting it, when an operating system generated initial sequence numbers in a guessable pattern (a largely solved problem now that modern operating systems randomise them properly). With a valid sequence number in hand, the attacker injects a spoofed packet that the receiver accepts as genuinely part of the session.
- Impact: session takeover and data injection, effectively bypassing whatever authentication happened only once, at the very start of the session.

TCP Hijacking: Mitigation

- Encrypt and authenticate the session itself (TLS), so an injected packet without the correct key is either rejected outright or immediately detected as tampered.
- Properly randomised initial sequence number generation, standard in modern operating systems, removes the prediction-based version of this attack entirely.
- Monitor for the anomalies hijacking tends to produce, such as unexpected duplicate ACKs or sequence numbers that jump in ways normal traffic wouldn't.
- This is exactly why later weeks on VPNs and cryptography matter here: encrypting end to end defeats hijacking regardless of whether the attacker can see or guess sequence numbers.

TCP RST Injection: Overview, Vulnerability & Technique

- Overview: the attacker forges a single RST packet, sent to one or both ends of an existing connection, to abruptly and prematurely terminate it.
- Vulnerability exploited: the same one as hijacking. A receiver accepts any RST that falls within the current valid sequence-number window as genuine, with no further verification.
- Technique: sniff or predict the current sequence number exactly as in hijacking, then inject a single forged RST rather than a stream of content. The mechanism is nearly identical to hijacking; the goal is different. Hijacking wants to take over and inject content, RST injection just wants the connection dead.
- Impact: denial of service against one specific, targeted connection rather than a whole host. This technique has historically been used at national scale for selective censorship, killing specific connections without blocking an entire address.

TCP RST Injection: Mitigation

- The same family of defences as hijacking: encrypted sessions and randomised sequence numbers remove the attacker's ability to forge a RST that the receiver will trust.
- Note the limit here: TLS protects the content of a session, but the underlying TCP connection itself can still be reset if the attacker forges a valid RST. Applications that need resilience against this often implement their own automatic reconnection logic above TCP.
- Monitoring for an unusual volume of connection resets, especially many at once from the same apparent pattern, is a useful detective signal at scale.

TCP SYN Flood: Overview, Vulnerability & Technique

- Overview: the attacker sends a high volume of SYN packets, usually with spoofed or random source addresses, and never completes the handshake.
- Vulnerability exploited: exactly the resource-reservation asymmetry from earlier in this lecture. The server commits real backlog memory the moment it receives a SYN, before it has any proof the client is genuine.
- Technique: because source addresses are typically spoofed, the server's SYN-ACK goes nowhere and no final ACK ever arrives. Each fake attempt sits in the half-open backlog until it times out. If the flood arrives faster than entries expire, the backlog fills completely.
- Impact: once the backlog is full, the server can no longer accept any new connection, legitimate or otherwise. Existing established connections are typically unaffected. It's specifically new connections that are denied.

TCP SYN Flood: Mitigation

- SYN cookies are the elegant fix: instead of allocating real backlog state on every SYN, the server encodes what it needs directly into the SYN-ACK's own sequence number, and only allocates real resources once the final ACK proves the client is genuine. This removes the resource asymmetry entirely rather than just tolerating it.
- Increasing backlog size and reducing the SYN-RECEIVED timeout raises the bar an attacker needs to clear, without eliminating the underlying weakness.
- Firewall and IPS-based SYN flood detection can filter or rate-limit incoming SYNs before they ever reach the server's backlog.
- For large-scale floods, dedicated DDoS protection services absorb volume no single server or firewall could handle alone, the same point made about volumetric attacks back in Week 2.

How Lightweight UDP Is

The same design choice, seen from the attacker's side

Just How Light UDP Actually Is

- No handshake at all. A UDP sender simply transmits, with no setup and no confirmation the receiver is even listening.
- No state maintained by the protocol itself. If an application needs to track a conversation, it has to build that logic itself, UDP won't do it for you.
- No reliability, no ordering, no flow control, no congestion control. Packets can be lost, duplicated, or arrive out of order, and UDP will neither notice nor care.
- An 8-byte header, versus TCP's 20-byte minimum. This is genuinely why UDP is the right choice for latency-sensitive, loss-tolerant traffic like DNS, VoIP, video, and gaming.
- Every one of these design choices is also exactly why UDP is comparatively easy to abuse. Less state and less verification means less for an attacker to work around.

Four UDP-Based Attacks at a Glance

Attack	Category	What it targets
UDP Port Scans	Information Gathering	Which UDP ports on a host are open or closed
UDP Host Sweeps	Information Gathering	Which hosts on a network are alive and reachable
UDP Bomb Attack	Header Abuse	Malformed packets crashing a poorly-validating stack
UDP Flood	Connection Resource Exhaustion (DoS)	Bandwidth and processing capacity

UDP Port Scans: Overview, Vulnerability & Technique

- Overview: the UDP equivalent of a TCP port scan, but structurally harder for the attacker, since UDP has no handshake to read a clear answer from.
- Vulnerability exploited: a closed UDP port typically triggers an ICMP Port Unreachable message, while an open port usually gives no response at all unless the specific application chooses to reply. Silence is genuinely ambiguous here; it could mean open, or it could just mean the probe was dropped somewhere.
- Technique: send UDP probes across a range of ports, treat ICMP Unreachable as closed, and treat silence as possibly open. To get a definitive answer on a specific port, the attacker often needs to send a payload a known service would actually respond to, such as a real DNS query on port 53.
- Impact: the same reconnaissance value as a TCP scan, just slower and noisier to obtain given the ambiguity involved.

UDP Port Scans: Mitigation

- Rate-limit ICMP Port Unreachable generation. This directly removes the scanner's clearest signal (a definite ‘closed’ answer) without breaking legitimate use of the protocol.
- IDS/IPS detection of UDP scan patterns, and minimising which UDP services are actually exposed, exactly as with TCP scanning.

UDP Host Sweeps: Overview, Vulnerability & Technique

- Overview: sending UDP probes across an address range to find live hosts, the UDP counterpart to the TCP host sweep.
- Vulnerability exploited: any host running a UDP service, or simply generating ICMP Port Unreachable for a closed one, reveals it's alive, again working around networks that block ICMP echo specifically.
- Mitigation: the same controls as UDP port scanning and TCP host sweeps together: rate-limit ICMP responses, detect horizontal scan patterns, minimise exposed services, and segment the network so one sweep can't see everything at once.

UDP Bomb Attack: Overview, Vulnerability & Technique

- Overview: a deliberately malformed UDP packet, crafted to crash or destabilise a target's network stack when it tries to process it.
- Vulnerability exploited: some stack implementations don't fully validate UDP header fields before acting on them, for example a length field that doesn't actually match the real payload size.
- Technique: craft a packet with an invalid or inconsistent field combination and send it directly to the target.
- Impact: denial of service through a crash or instability, conceptually the same legacy pattern as Week 4's Ping of Death and Teardrop attacks, just targeting UDP's own header instead of ICMP or IP fragmentation.

UDP Bomb Attack: Mitigation

- Keep operating systems and network device firmware patched. This exact class of vulnerability has largely been fixed industry-wide for years on properly maintained systems.
- Firewall and IPS-level packet validation can drop malformed UDP packets before they ever reach a potentially vulnerable host.
- As with every legacy malformed-packet attack this module has covered, the real remaining risk is unpatched embedded or IoT devices with long deployment lifespans.

UDP Flood: Overview, Vulnerability & Technique

- Overview: a high volume of UDP packets sent at a target to overwhelm its bandwidth or processing capacity, the direct UDP counterpart to Week 4's Ping Flood, but not limited to ICMP.
- Vulnerability exploited: UDP needs no connection setup at all, so an attacker can generate large volumes of traffic instantly and cheaply, and UDP has no built-in congestion control to throttle itself, or the attacker, back.
- Technique: flood packets at random or targeted ports on the victim. If a targeted port has no listening service, the victim may generate an ICMP Port Unreachable for every single packet, which becomes its own secondary resource drain on top of the flood itself.
- Impact: bandwidth and/or processing exhaustion, a straightforward denial of service, frequently combined with source address spoofing to hide the true origin.

UDP Flood: Mitigation

- Rate-limit UDP traffic at the perimeter, and filter or close any UDP port that doesn't need to be exposed at all.
- Disable or rate-limit ICMP Port Unreachable generation specifically to remove the secondary self-inflicted resource drain the flood can trigger.
- For large-scale floods, the same answer as everywhere else in this module: a dedicated DDoS protection service absorbs volume no on-site equipment can handle alone.
- A brief preview: many of the largest real-world UDP-based DDoS attacks (DNS, NTP, and memcached amplification) combine this flood pattern with an application-layer protocol that returns a much larger response than the request, turning modest attacker bandwidth into a massive flood at the victim. Those are application-layer abuses riding on UDP's spoofability, not covered in full detail here, but worth knowing this is exactly where UDP's lightness leads at scale.

Bringing It Together

What repeats across every attack today, and what to do about it

SUMMARY

Full Mitigation Comparison

Attack	Primary mitigation
TCP Port Scans	Minimise exposed services; scan-pattern detection; rate-limiting
TCP Host Sweeps	Minimise exposed services; scan-pattern detection; segmentation
OS Fingerprinting	Patching; stack normalisation; defence-in-depth
TCP Hijacking	TLS encryption; randomised sequence numbers; anomaly monitoring
TCP RST Injection	TLS encryption; randomised sequence numbers; reset-rate monitoring
TCP SYN Flood	SYN cookies; backlog tuning; SYN-rate filtering; DDoS protection
UDP Port Scans	ICMP Unreachable rate-limiting; scan-pattern detection
UDP Host Sweeps	ICMP Unreachable rate-limiting; segmentation
UDP Bomb Attack	Patching; firewall/IPS packet validation
UDP Flood	Rate-limiting; unnecessary port closure; DDoS protection

SUMMARY

Recurring Vulnerability Themes

- Layer 4, like every layer before it this module, was designed assuming a trusted network. Verification is minimal or absent by default at this layer itself.
- TCP's specific weakness is predictable, unauthenticated state: sequence numbers track a connection, but nothing cryptographically proves who's actually sending them, and the server commits real resources before verifying the other side at all.
- UDP's specific weakness is the complete absence of state to begin with. The same lightness that makes it efficient removes any opportunity to verify a sender before acting on what they send.
- Source address spoofing, introduced back in Week 4, is the thread running through nearly every attack today. Layer 4 has no mechanism of its own to verify where traffic genuinely came from; that job falls to IP-layer filtering or application-layer authentication instead.

SUMMARY

Layering Mitigations for High Security at Low Cost

- Not every effective control costs money. Some of the cheapest options available (disabling unused services, enabling SYN cookies, ingress/egress filtering) provide outsized protection relative to their cost, echoing Week 3's costing exercise.
- A sensible order: start free (disable what isn't needed, turn on SYN cookies, filter spoofed sources at the edge), then add low-cost configuration (firewall rate-limiting, IDS/IPS signatures on equipment you likely already own), and only reach for dedicated DDoS protection or deep packet inspection appliances where the risk genuinely justifies the expense.
- This is exactly the LO2 skill this whole module keeps returning to: a mitigation's value isn't just whether it works, it's whether it's proportionate to what you're actually protecting.

Summary

- TCP adds connection state, reliability, ordering, flow control, and congestion control on top of Layer 4's mandatory tasks. UDP adds none of it.
- The server commits real resources the moment a SYN arrives, before the handshake even completes. That single design decision is the root cause of SYN flood.
- TCP's six attacks split into reconnaissance (port scans, host sweeps, OS fingerprinting), session abuse (hijacking, RST injection, both exploiting the same unauthenticated sequence numbers), and resource exhaustion (SYN flood).
- UDP's lightness that makes it fast for DNS, VoIP, and gaming is the same property that makes it easy to scan around, flood, and spoof.
- Layered mitigation doesn't have to mean expensive mitigation. Several of today's strongest defences cost nothing beyond configuration time.

This Week's Lab & Tutorial

- Lab: Will be theory based with you developing a mitigation plan for a company
- Tutorial worksheet: study-notes-style questions covering Layer 4 fundamentals, the TCP handshake's resource implications, all ten attacks, and their mitigations. Use it to prep for the mid-semester test.

References

- Santos & Stuppi, CCNA Security 210-260 Official Cert Guide, 1st Ed., Cisco Press.
- Kaufman, Perlman, Speciner & Perlner, Network Security: Private Communication in a Public World, 3rd Ed., Prentice Hall.
- Cisco Networking Academy, CCNA Security curriculum (Transport layer security chapters).